Data Structures

BS (CS) _Fall_2024

Lab_09 Manual



Learning Objectives:

1. Stack

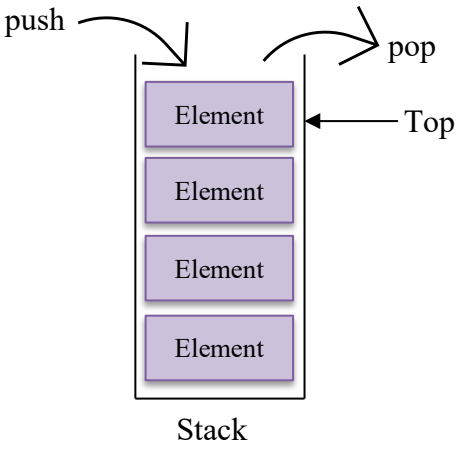
Stack and its Application

Stack:

Stack is a linear data structure based on **LIFO**(Last In First Out) principle in which the insertion of a new element and removal of an existing element takes place at the same end represented as the top of the stack. To implement the stack, it is required to maintain the pointer to the top of the stack , which is the last element to be inserted because we can access the elements only on the top of the stack.

Representation of Stack Data Structure:

The element that is inserted last will come out first. You can take a pile of plates kept on top of each other as a real-life example. The plate which we put last is on the top and since we remove the plate that is at the top, we can say that the plate that was put last comes out first.



Implement Stack using List

To implement a stack using an array, initialize an array and treat its end as the stack's top. Implement push (add to end), pop (remove from end), and peek (check end) operations, handling cases for an empty or full stack.

```
Initially, we set a Peek pointer to keep track of the topmost element of the stack.
Then the stack is initialized to -1 as its peek,
add (push) elements to the stack,
the peek gets updated to point to its topmost element,
and if we remove (pop) elements from the stack, the peek gets reduced.
```

```
class Node {
public:
    int data;
    Node* next;
    Node(int new_data) {
        this->data = new_data;
        this->next = nullptr;
    }
};
```

```
// Class to implement stack using a singly linked list
class Stack {

    // head of the linked list
    Node* head;

public:
    // Constructor to initialize the stack
    Stack() { this->head = nullptr; }
}
```

Types of Stack Data Structure

1. Fixed Size Stack

As the name suggests, a fixed size stack has a fixed size and cannot grow or shrink dynamically. If the stack is full and an attempt is made to add an element to it, an overflow error occurs. If the stack is empty and an attempt is made to remove an element from it, an underflow error occurs.

2. Dynamic Size Stack

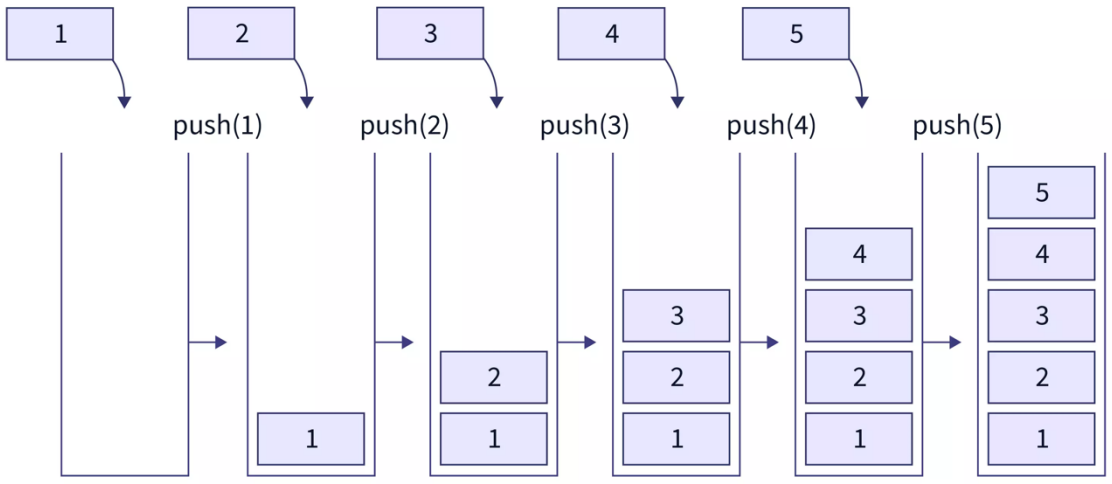
A dynamic size stack can grow or shrink dynamically. When the stack is full, it automatically increases its size to accommodate the new element, and when the stack is empty, it decreases its size. This type of stack is implemented using a linked list, as it allows for easy resizing of the stack.

Basic Operation on Stack:

There are various stack operations that are applicable on a stack. Stack operations are generally used to extract information and data from a stack data structure.
Some of the stack operations are given below.

1. Push

Adds an element to the top of the stack. If the stack is full, then it is said to be an **Overflow condition**.



Algorithm:

```
1. Initialise a node before pushing the element to the stack, we check if the stack is
full .
```

```
2. If the stack is full (top == capacity-1) , then Stack Overflows and we cannot insert
the element to the stack.
3. Otherwise, update the value of that node by data i.e. node->data = data
4. Now link this node to the top of the linked list
5. And update top pointer to the current node.
```

2. Pop

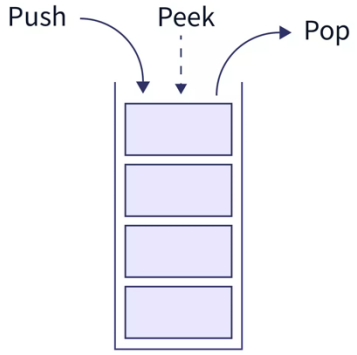
Removes the topmost element from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an **Underflow condition.**
The diagram below illustrates the concept.

Algorithm:

```
1. Before popping the element from the stack, we check if the stack is empty .
2. If the stack is empty (top == -1), then Stack Underflows and we cannot remove any
element from the stack.
3. Otherwise make pointer let say temp to the top node and move forward the top node by 1
step
4. Now free this temp node.
```

3. Peek

TopElement / Peek is a function in the stack which is used to extract the element present at the stack top.

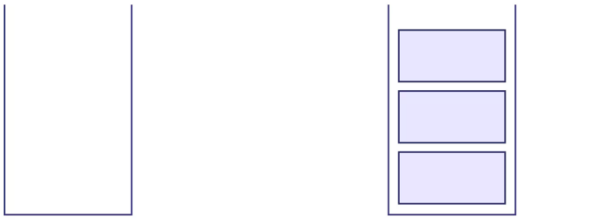


Algorithm:

```
1. Check if there is any node present or not, if not then return.
2. Otherwise return the value of top node of the linked list
```

4. Is Empty

isEmpty is a boolean function in stack definition which is used to check whether the stack is empty or not. It returns true if the stack is empty. Otherwise, it returns false.



Algorithm:

```
1. Check for the value of top in stack.
```

```
2. If (top == -1) , then the stack is empty so return true .
3. Otherwise, the stack is not empty so return false .
```

5. Is Full

isFull is a function which is used to check whether the stack has reached its maximum limit of insertion of data or not i.e. if 'maxLimit' is the maximum number of elements that can be stored in the stack and if there are exactly maxLimit number of elements present in the stack currently, then the function isFull() returns true. Otherwise, if the number of elements present in the stack currently are less than 'maxLimit', then isFull() returns false.

Algorithm:

```
1. Check for the value of top in stack.
2. If (top == capacity-1) , then the stack is full so return true .
3. Otherwise, the stack is not full so return false .
```

Advantage:

- **Dynamic memory allocation**: The size of the stack can be increased or decreased dynamically by adding or removing nodes from the linked list, without the need to allocate a fixed amount of memory for the stack upfront.
- **Efficient memory usage:** Since nodes in a singly linked list only have a next pointer and not a prev pointer, they use less memory than nodes in a doubly linked list.
- **Easy implementation**: Implementing a stack using a singly linked list is straightforward and can be done using just a few lines of code.

Application of Stack in C++

- **Expression Conversion**

1. Infix to Postfix
2. Infix to Prefix
3. Postfix to Infix
4. Prefix to Infix

- **Recursion: Call Stack**
  Recursive functions use something called the "call stack". When a program calls a function, the function is set at the top of the call stack. The last function call solves first as it is at the top of the stack, and the first function call is solved at last.
- **Memory Management: Stack allocation**
  The stack allocation happens on continuous blocks of the memory. It is known as **stack memory allocation** because the allocation is happening in the function call stack. The size of memory to be allocated is known to the compiler and whenever a function is called, its variables get memory allocated on the stack. The memory for the variables is de-allocated whenever the function call is over.
- **Compilation: Syntax Parsing**
  Many compilers use a stack data structure for parsing the syntax for expressions, blocks of code, etc., before translating the high-level code into low-level code.
- **Expression Evaluation: Parenthesis Checking**
  The stack data structure is used in expression evaluation as it keeps the ``record of the opening and closing parenthesis